AGENTS

AGENTS.md — Academic Triage System Backend

> **Master blueprint for the backend repository.** This document is the single source of truth that any AI agent, subagent, or developer must follow when working in this codebase. It defines the technology stack, hexagonal architecture layout, module responsibilities, naming conventions, and coding patterns for the Sistema de Triage y Gestión de Solicitudes Académicas.

---

1. Project Overview

Field	Value
Name	`academic-triage-system`
Purpose	Manage the full lifecycle of academic requests (registro, clasificación, priorización, asignación, atención, cierre) for the Programa de Ingeniería de Sistemas y Computación — Universidad del Quindío
Architecture	Hexagonal (Ports and Adapters) with DDD tactical patterns
API Contract	OpenAPI 3.0.3 — see `docs/openapi-academic-triage.yaml`
Auth	JWT Bearer Token with role-based access (ADMIN, STAFF, STUDENT)
AI Integration	Optional — Spring AI with OpenAI. System MUST work without it (RF-11)

---

2. Technology Stack

Runtime & Build

Technology	Version	Purpose
Java	21 (LTS)	Language — records, sealed interfaces, pattern matching, virtual threads
Spring Boot	3.5.x	Application framework
Gradle (Kotlin DSL)	8.x	Build tool — multi-module for hexagonal enforcement
MariaDB	11	Primary database
Docker / Docker Compose	Latest	Container orchestration for local dev and deployment

Core Dependencies

Dependency	Purpose	Module
`spring-boot-starter-web`	REST API (Tomcat embedded)	`infrastructure`
`spring-boot-starter-data-jpa`	ORM via Hibernate 6 / Jakarta Persistence	`infrastructure`
`spring-boot-starter-validation`	Bean validation (`jakarta.validation`)	`infrastructure`
`spring-boot-starter-security`	Authentication and authorization	`infrastructure`
`jjwt` (io.jsonwebtoken) 0.12.x	JWT creation and parsing	`infrastructure`
`spring-boot-starter-actuator`	Health checks, metrics	`bootstrap`

Dependency	Purpose	Module
Lombok 1.18.x	Reduces boilerplate in infrastructure layer (JPA entities, configs). **NEVER in domain**	`infrastructure`
MapStruct 1.6.x	Compile-time bean mapping (domain ↔ JPA entity ↔ DTO)	`infrastructure`
Spring AI 1.0.x	LLM integration (OpenAI chat client)	`infrastructure`
Flyway 10.x	Database migrations (versioned SQL scripts)	`infrastructure`
Springdoc OpenAPI 2.8.x	Swagger UI auto-generation from code	`infrastructure`

Testing Dependencies

Dependency	Purpose	Module
JUnit 5	Unit testing	ALL
AssertJ	Fluent assertions	ALL
Mockito	Mocking in adapter tests	`infrastructure`
`spring-boot-starter-test`	Spring test slices	`infrastructure`, `bootstrap`
Testcontainers (MariaDB)	Real DB in integration tests	`infrastructure`
ArchUnit 1.3.x	Architectural boundary enforcement	`infrastructure`
WireMock	External HTTP client testing (AI adapter)	`infrastructure`

Infrastructure Tools

Tool	Purpose
Docker Compose	MariaDB + app local environment
Adminer (optional)	Database GUI (lightweight, multi-DB)
Spotless (Gradle plugin)	Code formatting enforcement
JaCoCo	Test coverage reports

---

3. Repository Structure

```
academic-triage-system/
│
├── settings.gradle.kts                 # Multi-module declaration
├── build.gradle.kts                    # Root build — shared configs, plugins
├── gradle.properties                   # Version catalog
├── docker-compose.yml                  # MariaDB + Adminer
├── .env.example                        # Environment variables template
├── docs/
│   ├── openapi-academic-triage.yaml    # API contract (source of truth)
│   ├── diagram_classes.jpeg            # UML class diagram
│   └── diagram_state.png              # State machine diagram
│
├── domain/                             # 🟢 PURE JAVA — ZERO framework deps
│   ├── build.gradle.kts
│   └── src/
│       ├── main/java/co/edu/uniquindio/triage/domain/
│       │   ├── model/                  # Entities + Value Objects
│       │   │   ├── AcademicRequest.java         # Rich entity — aggregate root
│       │   │   ├── RequestHistory.java          # Entity (part of aggregate)
│       │   │   ├── User.java                    # Entity
│       │   │   ├── BusinessRule.java            # Entity
│       │   │   ├── RequestType.java             # Value object / small entity
│       │   │   ├── OriginChannel.java           # Value object / small entity
│       │   │   ├── RequestId.java               # Value object (record)
│       │   │   ├── UserId.java                  # Value object (record)
│       │   │   └── Email.java                   # Value object (record) self-
validating
│       │   ├── enums/
│       │   │   ├── RequestStatus.java           # Enum: REGISTERED, CLASSIFIED,
IN_PROGRESS, ATTENDED, CLOSED, CANCELLED, REJECTED
│       │   │   ├── Priority.java                # Enum: HIGH, MEDIUM, LOW
│       │   │   ├── Role.java                    # Enum: ADMIN, STAFF, STUDENT
│       │   │   └── HistoryAction.java           # Enum: REGISTERED, CLASSIFIED,
PRIORITIZED, ASSIGNED, ATTENDED, CLOSED, CANCELLED, REJECTED, INTERNAL_NOTE
│       │   ├── event/                  # Domain events (sealed interface + records)
│       │   │   └── RequestEvent.java
│       │   ├── service/                # Domain services (cross-aggregate logic ONLY)
│       │   │   ├── PriorityEngine.java          # Evaluates BusinessRules against a
request
│       │   │   └── StateTransitionValidator.java # Enforces valid status transitions
│       │   └── exception/              # Domain exceptions
│       │       ├── InvalidStateTransitionException.java
│       │       ├── RequestNotFoundException.java
│       │       ├── UserNotActiveException.java
│       │       └── BusinessRuleViolationException.java
│       └── test/java/co/edu/uniquindio/triage/domain/
│           ├── model/                  # Entity behavior tests
│           ├── service/                # Domain service tests
│           └── enums/                  # State transition tests
│
├── application/                        # 🔵 USE CASES + PORTS — minimal deps
│   ├── build.gradle.kts
│   └── src/
│       ├── main/java/co/edu/uniquindio/triage/application/
│       │   ├── port/
```

```
│       │   │   ├── in/                # Input ports (use case interfaces)
│       │   │   │   ├── request/
│       │   │   │   │   ├── CreateRequestUseCase.java
│       │   │   │   │   ├── ClassifyRequestUseCase.java
│       │   │   │   │   ├── PrioritizeRequestUseCase.java
│       │   │   │   │   ├── AssignRequestUseCase.java
│       │   │   │   │   ├── AttendRequestUseCase.java
│       │   │   │   │   ├── CloseRequestUseCase.java
│       │   │   │   │   ├── CancelRequestUseCase.java
│       │   │   │   │   ├── RejectRequestUseCase.java
│       │   │   │   │   └── GetRequestQuery.java
│       │   │   │   ├── auth/
│       │   │   │   │   ├── LoginUseCase.java
│       │   │   │   │   └── RegisterUseCase.java
│       │   │   │   ├── catalog/
│       │   │   │   │   ├── ManageRequestTypeUseCase.java
│       │   │   │   │   ├── ManageOriginChannelUseCase.java
│       │   │   │   │   └── ManageBusinessRuleUseCase.java
│       │   │   │   ├── user/
│       │   │   │   │   └── ManageUserUseCase.java
│       │   │   │   ├── ai/
│       │   │   │   │   ├── SuggestClassificationUseCase.java
│       │   │   │   │   └── GenerateSummaryUseCase.java
│       │   │   │   ├── report/
│       │   │   │   │   └── GetDashboardMetricsQuery.java
│       │   │   │   └── command/        # Self-validating command records
│       │   │   │       ├── CreateRequestCommand.java
│       │   │   │       ├── ClassifyRequestCommand.java
│       │   │   │       ├── PrioritizeRequestCommand.java
│       │   │   │       ├── AssignRequestCommand.java
│       │   │   │       ├── CloseRequestCommand.java
│       │   │   │       ├── CancelRequestCommand.java
│       │   │   │       ├── RejectRequestCommand.java
│       │   │   │       ├── LoginCommand.java
│       │   │   │       └── RegisterCommand.java
│       │   │   └── out/                # Output ports (driven interfaces)
│       │   │       ├── persistence/
│       │   │       │   ├── LoadRequestPort.java
│       │   │       │   ├── SaveRequestPort.java
│       │   │       │   ├── LoadUserPort.java
│       │   │       │   ├── SaveUserPort.java
│       │   │       │   ├── LoadRequestTypePort.java
│       │   │       │   ├── SaveRequestTypePort.java
│       │   │       │   ├── LoadOriginChannelPort.java
│       │   │       │   ├── SaveOriginChannelPort.java
│       │   │       │   ├── LoadBusinessRulePort.java
│       │   │       │   ├── SaveBusinessRulePort.java
│       │   │       │   └── RequestSearchPort.java
│       │   │       ├── security/
│       │   │       │   ├── PasswordEncoderPort.java
│       │   │       │   └── TokenProviderPort.java
│       │   │       └── ai/
│       │   │           └── AiAssistantPort.java
│       │   └── service/              # Use case implementations
│       │       ├── request/
│       │       │   ├── CreateRequestService.java
│       │       │   ├── ClassifyRequestService.java
```

```
│       │       │   ├── PrioritizeRequestService.java
│       │       │   ├── AssignRequestService.java
│       │       │   ├── AttendRequestService.java
│       │       │   ├── CloseRequestService.java
│       │       │   ├── CancelRequestService.java
│       │       │   ├── RejectRequestService.java
│       │       │   └── GetRequestService.java
│       │       ├── auth/
│       │       │   ├── LoginService.java
│       │       │   └── RegisterService.java
│       │       ├── catalog/
│       │       │   ├── RequestTypeService.java
│       │       │   ├── OriginChannelService.java
│       │       │   └── BusinessRuleService.java
│       │       ├── user/
│       │       │   └── UserManagementService.java
│       │       ├── ai/
│       │       │   ├── SuggestClassificationService.java
│       │       │   └── GenerateSummaryService.java
│       │       └── report/
│       │           └── DashboardMetricsService.java
│       └── test/java/co/edu/uniquindio/triage/application/
│           └── service/                # Use case tests with in-memory stubs
│
├── infrastructure/                     # 🟠 ADAPTERS + SPRING CONFIG
│   ├── build.gradle.kts
│   └── src/
│       ├── main/java/co/edu/uniquindio/triage/infrastructure/
│       │   ├── adapter/
│       │   │   ├── in/
│       │   │   │   └── rest/
│       │   │   │       ├── RequestController.java
│       │   │   │       ├── AuthController.java
│       │   │   │       ├── UserController.java
│       │   │   │       ├── CatalogController.java
│       │   │   │       ├── BusinessRuleController.java
│       │   │   │       ├── AiAssistantController.java
│       │   │   │       ├── ReportController.java
│       │   │   │       ├── dto/
│       │   │   │       │   ├── request/         # CreateRequestRequest,
RequestResponse, etc.
│       │   │   │       │   ├── auth/            # LoginRequest, RegisterRequest,
AuthResponse
│       │   │   │       │   ├── catalog/         # RequestTypeResponse,
OriginChannelResponse, etc.
│       │   │   │       │   ├── user/            # UserResponse
│       │   │   │       │   ├── ai/              # AiClassificationResponse,
AiSummaryResponse
│       │   │   │       │   ├── report/          # DashboardMetrics
│       │   │   │       │   └── common/          # PagedResponse, ErrorResponse
│       │   │   │       ├── mapper/
│       │   │   │       │   ├── RequestRestMapper.java      # MapStruct @Mapper
│       │   │   │       │   ├── UserRestMapper.java
│       │   │   │       │   └── CatalogRestMapper.java
│       │   │   │       └── advice/
│       │   │   │           └── GlobalExceptionHandler.java  # RFC 7807 ProblemDetail
│       │   │   └── out/
```

```
│       │   │       ├── persistence/
│       │   │       │   ├── entity/              # JPA entities (@Entity) — NOT domain
models
│       │   │       │   │   ├── AcademicRequestJpaEntity.java
│       │   │       │   │   ├── RequestHistoryJpaEntity.java
│       │   │       │   │   ├── UserJpaEntity.java
│       │   │       │   │   ├── BusinessRuleJpaEntity.java
│       │   │       │   │   ├── RequestTypeJpaEntity.java
│       │   │       │   │   └── OriginChannelJpaEntity.java
│       │   │       │   ├── repository/          # Spring Data JPA interfaces
│       │   │       │   │   ├── RequestJpaRepository.java
│       │   │       │   │   ├── UserJpaRepository.java
│       │   │       │   │   ├── BusinessRuleJpaRepository.java
│       │   │       │   │   ├── RequestTypeJpaRepository.java
│       │   │       │   │   └── OriginChannelJpaRepository.java
│       │   │       │   ├── adapter/             # Port implementations
│       │   │       │   │   ├── RequestPersistenceAdapter.java
│       │   │       │   │   ├── UserPersistenceAdapter.java
│       │   │       │   │   ├── RequestTypePersistenceAdapter.java
│       │   │       │   │   ├── OriginChannelPersistenceAdapter.java
│       │   │       │   │   ├── BusinessRulePersistenceAdapter.java
│       │   │       │   │   └── RequestSearchAdapter.java
│       │   │       │   ├── mapper/              # MapStruct domain ↔ JPA
│       │   │       │   │   ├── RequestPersistenceMapper.java
│       │   │       │   │   ├── UserPersistenceMapper.java
│       │   │       │   │   └── CatalogPersistenceMapper.java
│       │   │       │   └── specification/       # Spring Data JPA Specifications for
filters
│       │   │       │       └── RequestSpecifications.java
│       │   │       ├── security/
│       │   │       │   ├── JwtTokenAdapter.java          # Implements
TokenProviderPort
│       │   │       │   ├── BcryptPasswordAdapter.java     # Implements
PasswordEncoderPort
│       │   │       │   ├── JwtAuthenticationFilter.java
│       │   │       │   └── CustomUserDetailsService.java
│       │   │       └── ai/
│       │   │           └── SpringAiAssistantAdapter.java   # Implements
AiAssistantPort via Spring AI
│       │   └── config/
│       │       ├── BeanConfiguration.java           # Wires domain + application beans
│       │       ├── SecurityConfiguration.java       # Spring Security + JWT filter
chain
│       │       ├── PersistenceConfiguration.java    # JPA, Flyway, Hikari
│       │       ├── AiConfiguration.java             # Spring AI chat client
(conditional)
│       │       ├── WebConfiguration.java            # CORS, Jackson, etc.
│       │       └── OpenApiConfiguration.java        # Springdoc configuration
│       ├── main/resources/
│       │   └── db/migration/                        # Flyway SQL migrations
│       │       ├── V1__create_users_table.sql
│       │       ├── V2__create_request_types_table.sql
│       │       ├── V3__create_origin_channels_table.sql
│       │       ├── V4__create_academic_requests_table.sql
│       │       ├── V5__create_request_history_table.sql
│       │       ├── V6__create_business_rules_table.sql
│       │       └── V7__seed_initial_data.sql
```

```
│       └── test/java/co/edu/uniquindio/triage/infrastructure/
│           ├── adapter/in/rest/         # @WebMvcTest tests
│           ├── adapter/out/persistence/ # @DataJpaTest + Testcontainers
│           └── architecture/
│               └── ArchitectureTest.java  # ArchUnit rules
│
├── bootstrap/                          # 🔴 ENTRY POINT — wires everything
│   ├── build.gradle.kts
│   └── src/
│       ├── main/java/co/edu/uniquindio/triage/
│       │   └── Application.java         # @SpringBootApplication
│       ├── main/resources/
│       │   ├── application.yml
│       │   ├── application-dev.yml
│       │   ├── application-prod.yml
│       │   └── application-test.yml
│       └── test/java/co/edu/uniquindio/triage/
│           └── ApplicationIntegrationTest.java  # @SpringBootTest (sparingly)
│
└── skills/                             # AI agent skills (hexagonal-*)
    ├── hexagonal-architecture/SKILL.md
    ├── hexagonal-domain/SKILL.md
    ├── hexagonal-ports-adapters/SKILL.md
    ├── hexagonal-testing/SKILL.md
    └── hexagonal-spring-config/SKILL.md
```

---

4. Gradle Build Configuration

`settings.gradle.kts`

```kotlin
rootProject.name = "academic-triage-system"
include("domain", "application", "infrastructure", "bootstrap")
```

`gradle.properties` (Version Catalog)

```properties
# Java
javaVersion=21

# Spring
springBootVersion=3.5.9
springDependencyManagement=1.1.7
springAiVersion=1.0.0

# Libraries
lombokVersion=1.18.36
mapstructVersion=1.6.3
jjwtVersion=0.12.6
flywayVersion=10.22.0
springdocVersion=2.8.4
archunitVersion=1.3.0
testcontainersVersion=1.20.4
```

Root `build.gradle.kts`

```
plugins {
    java
    id("org.springframework.boot") version "${springBootVersion}" apply false
    id("io.spring.dependency-management") version "${springDependencyManagement}"
apply false
    id("com.diffplug.spotless") version "7.0.2"
}

subprojects {
    apply(plugin = "java")

    java {
        toolchain { languageVersion.set(JavaLanguageVersion.of(21)) }
    }

    repositories { mavenCentral() }

    tasks.withType<Test> { useJUnitPlatform() }
}
```

Module-Specific Builds

`domain/build.gradle.kts` — ZERO framework dependencies

```
plugins { `java-library` }

dependencies {
    testImplementation("org.junit.jupiter:junit-jupiter")
    testImplementation("org.assertj:assertj-core")
}
```

`application/build.gradle.kts` — Depends only on domain

```
plugins { `java-library` }

dependencies {
    implementation(project(":domain"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testImplementation("org.assertj:assertj-core")
}
```

`infrastructure/build.gradle.kts` — Spring lives here

```
plugins {
    `java-library`
    id("io.spring.dependency-management")
}

dependencyManagement {
    imports {
        mavenBom("org.springframework.boot:spring-boot-
dependencies:${springBootVersion}")
        mavenBom("org.springframework.ai:spring-ai-bom:${springAiVersion}")
    }
}

dependencies {
    implementation(project(":application"))

    // Spring Boot
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    implementation("org.springframework.boot:spring-boot-starter-validation")
    implementation("org.springframework.boot:spring-boot-starter-security")

    // JWT
    implementation("io.jsonwebtoken:jjwt-api:${jjwtVersion}")
    runtimeOnly("io.jsonwebtoken:jjwt-impl:${jjwtVersion}")
    runtimeOnly("io.jsonwebtoken:jjwt-jackson:${jjwtVersion}")

    // Spring AI (optional — enabled via profile)
    implementation("org.springframework.ai:spring-ai-openai-spring-boot-starter")

    // Database
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
    implementation("org.flywaydb:flyway-core")
    implementation("org.flywaydb:flyway-mysql")

    // Mapping
    implementation("org.mapstruct:mapstruct:${mapstructVersion}")
    annotationProcessor("org.mapstruct:mapstruct-processor:${mapstructVersion}")

    // Lombok (infrastructure ONLY)
    compileOnly("org.projectlombok:lombok:${lombokVersion}")
    annotationProcessor("org.projectlombok:lombok:${lombokVersion}")
    annotationProcessor("org.projectlombok:lombok-mapstruct-binding:0.2.0")

    // Documentation
    implementation("org.springdoc:springdoc-openapi-starter-webmvc-
ui:${springdocVersion}")

    // Testing
    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.springframework.security:spring-security-test")
    testImplementation("org.testcontainers:mariadb:${testcontainersVersion}")
    testImplementation("org.testcontainers:junit-jupiter:${testcontainersVersion}")
    testImplementation("com.tngtech.archunit:archunit-junit5:${archunitVersion}")
    testImplementation("org.wiremock:wiremock-standalone:3.10.0")
}
```

`bootstrap/build.gradle.kts` — Entry point

```
plugins {
    id("org.springframework.boot")
    id("io.spring.dependency-management")
}

dependencies {
    implementation(project(":infrastructure"))
    implementation("org.springframework.boot:spring-boot-starter")
    implementation("org.springframework.boot:spring-boot-starter-actuator")

    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.testcontainers:mariadb")
}
```

---

5. Docker Configuration

`docker-compose.yml`

```yaml
services:
  mariadb:
    image: mariadb:11
    container_name: triage-mariadb
    environment:
      MARIADB_DATABASE: triage_db
      MARIADB_USER: ${DB_USER:-triage_user}
      MARIADB_PASSWORD: ${DB_PASS:-triage_secret}
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASS:-root_secret}
    ports:
      - "3306:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 5s
      retries: 5

  adminer:
    image: adminer:latest
    container_name: triage-adminer
    ports:
      - "8081:8080"
    depends_on:
      mariadb:
        condition: service_healthy

volumes:
  mariadb_data:
```

`.env.example`

```
# Database
DB_USER=triage_user
DB_PASS=triage_secret
DB_ROOT_PASS=root_secret
DB_URL=jdbc:mariadb://localhost:3306/triage_db

# JWT
JWT_SECRET=your-256-bit-secret-change-in-production
JWT_EXPIRATION_MS=86400000

# Spring AI (optional — leave empty to disable)
SPRING_AI_OPENAI_API_KEY=
SPRING_AI_OPENAI_CHAT_OPTIONS_MODEL=gpt-4o-mini
```

---

6. Application Configuration

`bootstrap/src/main/resources/application.yml`

```yaml
server:
  port: 8080
  error:
    include-message: always

spring:
  application:
    name: academic-triage-system
  threads:
    virtual:
      enabled: true
  datasource:
    url: ${DB_URL:jdbc:mariadb://localhost:3306/triage_db}
    username: ${DB_USER:triage_user}
    password: ${DB_PASS:triage_secret}
    driver-class-name: org.mariadb.jdbc.Driver
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.MariaDBDialect
  flyway:
    enabled: true
    locations: classpath:db/migration
  jackson:
    default-property-inclusion: non_null
    serialization:
      write-dates-as-timestamps: false
  ai:
    openai:
      api-key: ${SPRING_AI_OPENAI_API_KEY:}
      chat:
        options:
          model: ${SPRING_AI_OPENAI_CHAT_OPTIONS_MODEL:gpt-4o-mini}
          temperature: 0.3

# JWT
app:
  jwt:
    secret: ${JWT_SECRET:default-dev-secret-change-me}
    expiration-ms: ${JWT_EXPIRATION_MS:86400000}

# Springdoc
springdoc:
  api-docs:
    path: /api-docs
  swagger-ui:
```

```yaml
    path: /swagger-ui.html
    tags-sorter: alpha
    operations-sorter: method
```

---

7. Domain Model — Behavioral Design

Request Lifecycle (State Machine)

```
              ┌──────────────┐
              │  REGISTERED  │
              └──────┬───────┘
                     │ classify()
              ┌──────▼───────┐
              │  CLASSIFIED  │──────────┐
              └──────┬───────┘          │ cancel()
                     │ assign()         │
              ┌──────▼───────┐    ┌─────▼────────┐
              │ IN_PROGRESS  │    │ CANCELLED    │ ◀── Terminal
              └──────┬───────┘    └──────────────┘
                     │ attend()
              ┌──────▼───────┐
              │   ATTENDED   │
              └──────┬───────┘
                     │ close()
              ┌──────▼───────┐
              │    CLOSED    │ ◀── Terminal
              └──────────────┘

    REGISTERED ──reject()──▶ REJECTED (Terminal)
    REGISTERED ──cancel()──▶ CANCELLED (Terminal)
```

Valid Transitions (enforced in `AcademicRequest` entity)

From	To	Method	Required Role
REGISTERED	CLASSIFIED	`classify()`	STAFF
CLASSIFIED	IN_PROGRESS	`assign()`	STAFF
IN_PROGRESS	ATTENDED	`attend()`	STAFF
ATTENDED	CLOSED	`close()`	STAFF
REGISTERED	CANCELLED	`cancel()`	STUDENT (owner), STAFF, ADMIN
CLASSIFIED	CANCELLED	`cancel()`	STUDENT (owner), STAFF, ADMIN
REGISTERED	REJECTED	`reject()`	ADMIN

Domain Rules

1. **AcademicRequest** is the aggregate root. It contains `RequestHistory` entries (composition).
2. Every state transition creates a `RequestHistory` entry automatically.
3. Terminal states (CLOSED, CANCELLED, REJECTED) make the request **immutable**.
4. `close()` requires a non-empty `closingObservation`.
5. `cancel()` requires a non-empty `cancellationReason`.
6. `reject()` requires a non-empty `rejectionReason`.
7. `assign()` validates the target user is active and has STAFF role.
8. `PriorityEngine` evaluates active `BusinessRule` entities against request data to suggest priority.

9. Priority assignment requires a non-empty `justification`.

Lombok Policy

Module	Lombok Allowed?	Reason
`domain`	**NO**	Domain must be pure Java. Use records for value objects. Write constructors and getters explicitly.
`application`	**NO**	Use records for commands/queries. Services have few fields — manual constructor.
`infrastructure`	**YES**	JPA entities (`@Builder`, `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`), config DTOs.
`bootstrap`	Minimal	Only if needed for config properties.

MapStruct Policy

- MapStruct mappers live **exclusively** in `infrastructure` (both `adapter/in/rest/mapper/` and `adapter/out/persistence/mapper/`).
- Use `componentModel = "spring"` for all `@Mapper` interfaces.
- Configure `lombok-mapstruct-binding` to enable MapStruct + Lombok coexistence on JPA entities.
- Domain model ↔ JPA entity mapping: `OrderPersistenceMapper`
- Domain model ↔ REST DTO mapping: `OrderRestMapper`
- **NEVER** map directly from JPA entity to DTO. Always go through domain: `JPA Entity → Domain → DTO`.

---

8. Security Design

JWT Flow

1. User calls `POST /api/v1/auth/login` with username + password.
2. `AuthController` → `LoginUseCase` → validates via `PasswordEncoderPort` → generates token via `TokenProviderPort`.
3. Token returned in `AuthResponse` body.
4. All subsequent requests include `Authorization: Bearer <token>`.
5. `JwtAuthenticationFilter` (OncePerRequestFilter) intercepts, validates, and sets `SecurityContext`.

Role-Based Access Matrix

Endpoint	STUDENT	STAFF	ADMIN
`POST /requests`	✅	✅	❌
`GET /requests`	Own only	All	All
`PATCH /requests/{id}/classify`	❌	✅	❌
`PATCH /requests/{id}/prioritize`	❌	✅	❌
`PATCH /requests/{id}/assign`	❌	✅	❌
`PATCH /requests/{id}/attend`	❌	✅	❌
`PATCH /requests/{id}/close`	❌	✅	❌
`PATCH /requests/{id}/cancel`	Owner	✅	✅
`PATCH /requests/{id}/reject`	❌	❌	✅
`POST /ai/classify`	✅	✅	✅
`GET /ai/summary/{id}`	Owner	✅	✅
Catalog CRUD	❌	❌	✅
User management	❌	❌	✅
Business rules	❌	❌	✅

9. AI Integration (RF-09, RF-10, RF-11)

Architecture

```
Controller → SuggestClassificationUseCase → AiAssistantPort ← SpringAiAssistantAdapter
```

- `AiAssistantPort` is an **output port** in `application/port/out/ai/`.
- `SpringAiAssistantAdapter` in `infrastructure/adapter/out/ai/` uses Spring AI's `ChatClient`.
- When `SPRING_AI_OPENAI_API_KEY` is empty, a **NoOpAiAssistantAdapter** is wired via `@ConditionalOnProperty`, returning empty optionals.
- The system is **100% functional without AI** (RF-11).

Conditional Wiring

```java
// infrastructure/config/AiConfiguration.java
@Configuration
public class AiConfiguration {

    @Bean
    @ConditionalOnProperty(name = "spring.ai.openai.api-key", havingValue = "",
matchIfMissing = true)
    AiAssistantPort noOpAiAssistant() {
        return new NoOpAiAssistantAdapter();
    }

    @Bean
    @ConditionalOnProperty(name = "spring.ai.openai.api-key")
    AiAssistantPort springAiAssistant(ChatClient.Builder chatClientBuilder) {
        return new SpringAiAssistantAdapter(chatClientBuilder.build());
    }
}
```

---

10. Testing Strategy

Layer	Test Count Target	Speed Target	Tooling
Domain unit tests	High	< 1s total	JUnit 5 + AssertJ
Application use case tests	High	< 3s total	JUnit 5 + AssertJ + in-memory stubs
REST adapter tests	Medium	< 10s total	@WebMvcTest + MockitoBean
Persistence adapter tests	Medium	< 30s total	@DataJpaTest + Testcontainers
AI adapter tests	Low	< 5s total	WireMock
Architecture tests	1 class	< 2s total	ArchUnit
Full integration	Few critical paths	< 60s total	@SpringBootTest + Testcontainers

ArchUnit Rules (Mandatory)

```java
@AnalyzeClasses(packages = "co.edu.uniquindio.triage")
class ArchitectureTest {

    @ArchTest static final ArchRule hexagonal = onionArchitecture()
            .domainModels("..domain.model..", "..domain.enums..", "..domain.event..")
            .domainServices("..domain.service..", "..domain.exception..")
            .applicationServices("..application..")
            .adapter("rest", "..infrastructure.adapter.in.rest..")
            .adapter("persistence", "..infrastructure.adapter.out.persistence..")
            .adapter("security", "..infrastructure.adapter.out.security..")
            .adapter("ai", "..infrastructure.adapter.out.ai..")
            .adapter("config", "..infrastructure.config..");

    @ArchTest static final ArchRule domainIsFrameworkFree =
            noClasses().that().resideInAPackage("..domain..")
                    .should().dependOnClassesThat()
                    .resideInAnyPackage("org.springframework..", "jakarta..",
"lombok..");

    @ArchTest static final ArchRule applicationIsFrameworkFree =
            noClasses().that().resideInAPackage("..application..")
                    .should().dependOnClassesThat()
                    .resideInAnyPackage("org.springframework..", "jakarta..",
"lombok..");
}
```

---

11. Naming Conventions

Concept	Pattern	Example
Input port (write)	`*UseCase`	`CreateRequestUseCase`
Input port (read)	`*Query`	`GetRequestQuery`
Output port	`*Port` or `*Gateway`	`LoadRequestPort`, `AiAssistantPort`
Command record	`*Command`	`CreateRequestCommand`
Use case service	`*Service`	`CreateRequestService`
REST controller	`*Controller`	`RequestController`
REST DTO	`*Request` / `*Response`	`CreateRequestRequest`, `RequestResponse`
JPA entity	`*JpaEntity`	`AcademicRequestJpaEntity`
Spring Data repo	`*JpaRepository`	`RequestJpaRepository`
Adapter impl	`*Adapter`	`RequestPersistenceAdapter`
Mapper	`*Mapper`	`RequestPersistenceMapper`, `RequestRestMapper`
Domain exception	Descriptive name	`InvalidStateTransitionException`
Flyway migration	`V{n}__{description}.sql`	`V1__create_users_table.sql`
Package base	`co.edu.uniquindio.triage`	—

---

12. Coding Standards

General

- **Java 21 features**: Use `var` for local variables, records for immutable data, sealed interfaces for closed hierarchies, pattern matching with switch.
- **No `null` returns**: Use `Optional` or throw domain exceptions.
- **No field injection**: Constructor injection everywhere (`final` fields).
- **Immutability**: Domain objects are immutable or have controlled mutation via business methods.
- **Package-private by default**: Adapter classes, JPA entities, and mappers should be package-private. Only ports and domain models are `public`.

REST API

- Base path: `/api/v1`
- JSON only (`application/json`)
- Errors: RFC 7807 `ProblemDetail` via `@RestControllerAdvice`
- Pagination: `page`, `size`, `sort` parameters — Spring Data `Pageable`
- HTTP status codes: `201` for creation, `200` for success, `400`/`401`/`403`/`404`/`409` for errors

Database

- Schema managed by Flyway — **never** `ddl-auto: update` or `create`
- Table names: `snake_case` plural (`users`, `academic_requests`, `request_history`)
- Column names: `snake_case` (`registration_date_time`, `assigned_to_id`)
- Indexes on foreign keys and frequently filtered columns (`status`, `priority`)

---

13. Registered Skills

Skill	Description	Location
`hexagonal-architecture`	Project structure, Dependency Rule, multi-module setup	SKILL.md
`hexagonal-domain`	Rich entities, records, sealed interfaces, domain events	SKILL.md
`hexagonal-ports-adapters`	Ports, adapters, mappers, DTOs, exception handling	SKILL.md
`hexagonal-testing`	Test pyramid, stubs, @WebMvcTest, Testcontainers, ArchUnit	SKILL.md
`hexagonal-spring-config`	Bean wiring, virtual threads, profiles, GraalVM, app.yml	SKILL.md

---

14. Quick Start

```shell
# 1. Clone and enter the project
git clone <repo-url> && cd academic-triage-system

# 2. Copy environment variables
cp .env.example .env

# 3. Start MariaDB
docker compose up -d mariadb

# 4. Run the application
./gradlew bootstrap:bootRun

# 5. Open Swagger UI
open http://localhost:8080/swagger-ui.html

# 6. Run all tests
./gradlew test

# 7. Run only domain tests (milliseconds)
./gradlew domain:test

# 8. Run architecture tests
./gradlew infrastructure:test --tests "*ArchitectureTest*"

# 9. Verify domain has zero Spring deps
./gradlew domain:dependencies --configuration compileClasspath
```